Speaker Embedding Training Project -- Complete Development Report

Project Goal

Build a speaker embedding training system from scratch -- not following any existing architecture (ECAPA-TDNN, etc.) -- with a custom causal Conv-LSTM-Attention encoder, gender-aware training diagnostics via EMA memory bank, and a full training/inference pipeline.

Phase 1: Foundation (Commits 624ecf8 -> d04a4e0)

What was built

- Minimal training pipeline: single train.py with inline model, dataset, loss
- create_manifest.py for building data manifests from dataset folders
- plot_manifest.py for 8-panel data distribution visualization

Key decisions & reasoning

Manifest-based data loading -- We chose a JSON manifest approach (create_manifest.py) so any dataset structure can be supported. Each entry stores audio_file_path, speaker_id, duration, dataset_name, language, gender. Gender file is mandatory because the entire diagnostic system depends on gender-aware analysis.

Gender handling -- Started with gender as optional, but quickly made it mandatory after realizing the EMA diagnostic system (planned from the start) requires gender labels. Multiple commits refined gender file loading (JSON/CSV/TSV), validation, and error reporting.

Combined loss (AAM + Prototypical) -- We discussed which loss function to use. Decision: AAMSoftmax provides strong classification signal, Prototypical loss teaches metric learning. Combined loss (0.7 AAM + 0.3 Proto) gets benefits of both. ContrastiveLoss (NT-Xent) was also implemented as an option.

SpeakerBatchSampler with gender balance -- Each batch has exactly speakers_per_batch/2 male + speakers_per_batch/2 female. This ensures both genders are equally represented in every training step, which is critical for fair diagnostic comparisons later.

LR warmup + gradient clipping + AMP -- Standard training stability measures. Linear warmup prevents early divergence, grad clipping (default 5.0) prevents explosions, AMP for memory/speed efficiency.

Phase 2: Architecture & Training (Commits 2b40d42 -> a6a7267)

What was built

- Separated code into model.py, losses.py, dataset.py
- Custom causal Conv-LSTM-Attention encoder (~6.25M params)
- Step-based training (replacing epoch-based)

Key decisions & reasoning

Causal architecture -- User specifically requested a causal model (no future context), suitable for streaming/online applications. The design:

- 3 Conv1d layers with left-only padding (causal) -- no padding='same', explicit nn.functional.pad(x, (pad, 0))
- 2-layer LSTM (inherently causal)
- Attention pooling over time
- Linear projection (512 -> 256) + BatchNorm

~6M params target -- User wanted a specific size, not too large, not too small. The architecture was designed to hit ~6.25M params with default settings.

Step-based training -- Switched from epoch-based to step-based because: (1) dataset size varies, step-based gives consistent training length, (2) infinite data loader with reshuffling gives better randomization, (3) validation/checkpointing at fixed step intervals is more predictable.

Code separation -- model.py, losses.py, dataset.py split from monolithic train.py for maintainability and testing.

Phase 3: EMA Memory Bank & Diagnostics (Commits 7eeedbd -> ad80c63)

What was built

- ema_bank.py -- EMA Memory Bank tracking per-speaker embeddings
- 4 diagnostic metrics: M-self, F-self, M-mix, F-mix
- All metrics logged to TensorBoard

Key decisions & reasoning

The core idea -- Track how well the model learns male vs female speakers separately. The EMA bank maintains a running average embedding for each speaker. By comparing batch embeddings against bank embeddings, we can measure learning quality per gender.

4 diagnostic metrics (all cosine similarity):

Metric	What it measures	Good direction
M-self	Male batch emb vs male bank emb similarity	Higher = better (model is consistent)
F-self	Female batch emb vs female bank emb similarity	Higher = better
M-mix	Pairwise similarity among male bank speakers	Lower = better (speakers are distinct)
F-mix	Pairwise similarity among female bank speakers	Lower = better

Self metrics tell you "is the model producing consistent embeddings for known speakers?" If M-self is low but F-self is high, the model learns females better than males.

Mix metrics tell you "are different speakers being mapped to distinct points?" If M-mix is high, male speakers are collapsing to similar embeddings -- the model can't distinguish them.

EMA smoothing -- Raw diagnostics are noisy. We apply EMA smoothing (alpha=0.05) to get stable trend lines.

Cold speaker filtering -- Speakers not seen in the last 500 steps are excluded from mix computation. Their bank embeddings are stale and would corrupt the metric.

Bug fix: mix metric direction -- Initially M-mix/F-mix computed cosine distance (1 - similarity). But the interpretation should be: high similarity = bad (speakers collapsing). Fixed to report raw cosine similarity directly.

Bug fix: self metric direction -- Initially used cosine distance. Changed to cosine similarity for consistency -- all 4 metrics now report cosine similarity.

Ratio recommendation removed -- Initially implemented a system that would recommend adjusting the male/female batch ratio based on diagnostic scores. Removed because: (1) adds complexity, (2) the diagnostic signals alone are valuable, (3) ratio adjustment can be added back later when we understand the diagnostics better.

Phase 4: Robustness & Stability (Commits 224bf06 -> 614aa10)

What was built

- NaN safeguards across the pipeline
- Code snapshot for reproducibility

Key decisions & reasoning

NaN prevention (multiple layers):

1. Short audio: Loop-repeat instead of zero-padding (silence -> log(0) = -inf -> NaN)
2. Log mel: torch.log(mel + 1e-9) instead of torch.log(mel)
3. Cosine similarity: eps=1e-8 in all F.normalize calls
4. AAMSoftmax: Clamp cosine to [-1+1e-7, 1-1e-7] before acos (prevents NaN at boundaries)
5. ContrastiveLoss: logsumexp instead of exp + log (temperature=0.07 makes exp overflow)
6. Training loop: Skip step on NaN/Inf loss or gradients, log to TensorBoard

NaN skip + scheduler sync -- When skipping a NaN step, we still call scheduler.step() to keep the LR schedule synchronized. Without this, the warmup/decay curve shifts.

Code snapshot -- Saves all .py, .json, .yaml files as a timestamped zip at training start. Placed after sanity check (1 train batch + 1 validation) so the snapshot only happens if the code actually works.

Phase 5: Training Features (Commits fab1375 -> 02c24a4)

What was built

- LR finder (sweep LR range, find optimal)
- Keep-last-N checkpoint management
- Augmentation pipeline (augmentation.py)
- Descriptive experiment naming for TensorBoard

Key decisions & reasoning

LR finder -- Sweeps LR from 1e-7 to 1.0 exponentially over 100 steps, finds the steepest loss decrease point. Restores model state after. Optional (lr_finder: false by default), only runs on fresh start (not resume). Can auto-apply or just suggest.

Keep-last-N checkpoints -- Saves checkpoint_step_{N}.pt every save_interval steps, keeps only last 3 (configurable). Never deletes best_model.pt. Prevents disk bloat on long training runs.

Augmentation pipeline -- Referenced 3D-Speaker augmentation approach. Created standalone functions + wrapper classes:

- WavAugmentor: noise addition, reverberation, speed perturbation

- SpecAugmentor: frequency masking, time masking

- All probabilities default to 0.0 -- augmentation is off until explicitly enabled. This lets training start clean and augmentation can be tuned later.

- Applied to train set only, never validation.

Descriptive experiment names -- Auto-generated from config: convlstm_attn_combined_emb256_spk100x4_lr0.001_warm1000_noaug_steps100k. Makes TensorBoard runs self-documenting.

Phase 6: Comprehensive Codebase Review (Commit 2ed35dd)

What was fixed

After the core system was working and training was underway, we did a thorough code review across all files. Found and fixed ~10 issues:

File	Fix	Why
losses.py	logsumexp in ContrastiveLoss	Naive exp(sim/0.07) overflows float32
augmentation.py	.cpu() before .numpy()	Would crash on GPU tensors
augmentation.py	Dimension guard in spec_augment	Edge case: n_mels=1 or T=1
model.py	bias=False on projection	Embeddings get L2-normalized; bias shifts off unit sphere
dataset.py	Try-except audio loading + retry	Corrupted files don't crash training
dataset.py	Removed dead _pick_speakers()	Unused code
train.py	GradScaler(device.type) not "cuda"	Works on CPU too
train.py	scheduler.step() in NaN skip	Keeps LR schedule in sync
train.py	Cache all_params list	Avoid rebuilding every step
train.py	Seed control	random.seed, torch.manual_seed, np.random.seed
train.py	worker_init_fn on DataLoaders	Reproducible augmentation across workers
train.py	Log grad_norm to TensorBoard	Monitor gradient health
train.py	Copy config to output_dir	Reproducibility
train.py	Validate speakers_per_batch vs manifest	Prevent runtime crash
config.json	Added "seed": 42	Reproducibility default

Phase 7: Inference & Final Simplification (Commits 1ad172c -> dadf17c)

What was built

- inference.py -- Batch inference + speaker verification

- Simplified diagnostics to pure cosine similarity

Key decisions & reasoning

Inference design -- Two modes:

1. Extract: Load directory of audio -> pad/crop all to target duration (default 10s) -> batch forward -> save L2-normalized embeddings as .pt

2. Verify: Load two audio files -> extract embeddings -> cosine similarity -> same/different decision

Fixed duration for batching -- All audio normalized to target_duration (default 10s) via loop-repeat or crop. This ensures all features have the same time dimension, enabling torch.stack for batch processing. Training uses 3s segments, but inference uses 10s for better quality.

Feature extraction inline -- Not importing from dataset.py to avoid label/augmentation dependencies. Inference needs only: load -> resample -> mono -> pad/crop -> mel -> log -> model.

Diagnostic simplification -- Removed all ratio recommendation logic (thresholds, options, _compute_recommended_ratio(), set_ratio() in sampler). The 4 cosine similarity diagnostics remain as pure monitoring. Ratio can be re-added later when the diagnostic signals are better understood from real training runs.

Final File Inventory

File	Lines	Purpose
model.py	~35	Causal Conv-LSTM-Attention encoder (6.25M params)
losses.py	~120	AAMSoftmax, Prototypical, Contrastive, Combined losses
dataset.py	~175	SpeakerDataset + gender-balanced SpeakerBatchSampler
ema_bank.py	~230	EMA Memory Bank + 4 cosine similarity diagnostics
augmentation.py	~175	Noise, reverb, speed, SpecAugment (all default off)
train.py	~770	Full training loop with all features
inference.py	~165	Batch extraction + speaker verification
config.json	~55	Complete configuration
verify.py	~100	Component sanity checks
create_manifest.py	~130	Multiprocessed manifest builder
plot_manifest.py	~200	8-panel data visualization
.gitignore	4	Cache, experiments, images

Config Summary

```
Model:      Conv-LSTM-Attention, 256-dim embeddings, ~6.25M params
Loss:       Combined (0.7 AAM + 0.3 Proto), margin=0.2, scale=30
Training:   100K steps, lr=0.001, warmup=1000, cosine decay
Batch:      100 speakers x 4 samples = 400 samples/batch (50M + 50F)
Features:   80-mel, 512 FFT, 160 hop, 400 win, 3s segments
EMA:        alpha=0.01, warmup=2000 steps, diagnostics every 100 steps
Augment:    All disabled (prob=0.0), ready to enable
Inference:  10s target duration, batch_size=32
```

TensorBoard Metrics (20+)

Training: loss, accuracy, lr, ms_per_step, grad_norm, nan_count

Validation: loss, accuracy

Diagnostics: m_self, f_self, m_mix, f_mix (raw + EMA-smoothed)

Bank: initialized_speakers

LR Finder: loss, lr (if enabled)

Key Design Principles

1. Gender-aware from day one -- Batch sampling, diagnostics, validation split all gender-balanced
2. Causal architecture -- Suitable for streaming/online speaker verification
3. Defense in depth for NaN -- 6 layers of NaN prevention
4. Augmentation-ready but off -- All augmentation coded and integrated, zero probability by default
5. Diagnostic monitoring without intervention -- Log everything, apply nothing automatically
6. Reproducibility -- Seeds, config snapshots, code snapshots, deterministic workers
7. Step-based, not epoch-based -- Consistent training regardless of dataset size

---

38 commits | 12 files | ~2,160 lines of code